

Databricks Cluster Planning & Compute Infrastructure Strategy

Optimized Cluster Sizing, Autoscaling, Memory, Workload Classification and Runtime Management for 700 Python + 300 PySpark Jobs

Scope & Compute Strategy Summary

Workload Volume
Total: ~1000 Jobs
Python: ~700 | PySpark: ~300
Environments: DEV, SIT, UAT, PROD

Compute Model
Workload-based Job Clusters
Serverless: lightweight Python only
Classic: libraries, drivers, Spark

All-Purpose Cluster
DEV / debugging only
Notebook testing, sample data
Unit testing

▲ PROD Guardrail
No PROD batch execution on all-purpose clusters
Enforced via cluster policies

★ Key Recommendation
Use multiple right-sized job clusters
Instead of one shared large cluster

Workload Classification → Cluster Selection → Cluster Sizing → Policy & Runtime Control → Monitoring & Optimization

1. Workload Classification

A. Lightweight Python Control
Count: 400-500
Input: control/metadata/config/lookups
Output: audit, status, control writes, logs
Data: < 1 GB
Compute: Serverless first
Fallback: Small Classic Job Cluster

B. Python Validation / Reconciliation
Count: 150-200
Input: CSV, S3 files, SQL extracts, refs
Output: validation, exceptions, recon reports
Data: 1 GB - 20 GB
Compute: Small Classic Job Cluster

C. Medium PySpark Transformation
Count: 150-180
Input: S3 landing, Delta, MSSQL/PG extracts
Output: staging, curated Delta, pre-land
Data: 20 GB - 200 GB
Compute: Medium PySpark Job Cluster

D. Heavy PySpark / Delta Merge
Count: 60-100
Input: large Delta, historical, incrementals
Output: merge, publish, egress datasets
Data: 200 GB - 1 TB
Compute: Large PySpark Job Cluster

E. Backfill / Critical Batch
Count: 10-20
Input: historical, multi-source joins, merges
Output: publish, history, large outputs
Data: 1 TB+
Compute: XL Batch Job Cluster

1. Databricks Compute Type

1. Serverless Jobs Compute
Use For:
Lightweight Python, metadata, audit, control, small validation
Sizing:
Driver / Workers / Autoscale: Managed
Memory: Managed
Decision Rule:
Validate secrets, connectivity, libraries, UC access, logging first

1. Small Classic Job Cluster
Use For:
Python + custom libs, JDBC/ODBC, MSSQL/PG, recon, small file compare
Sizing:
Driver: 4-8 vCPU, 16-32 GB
Worker: 4-8 vCPU, 16-32 GB
Workers: fixed 1 or min/max 4
Auto-Termination: 10-15 min

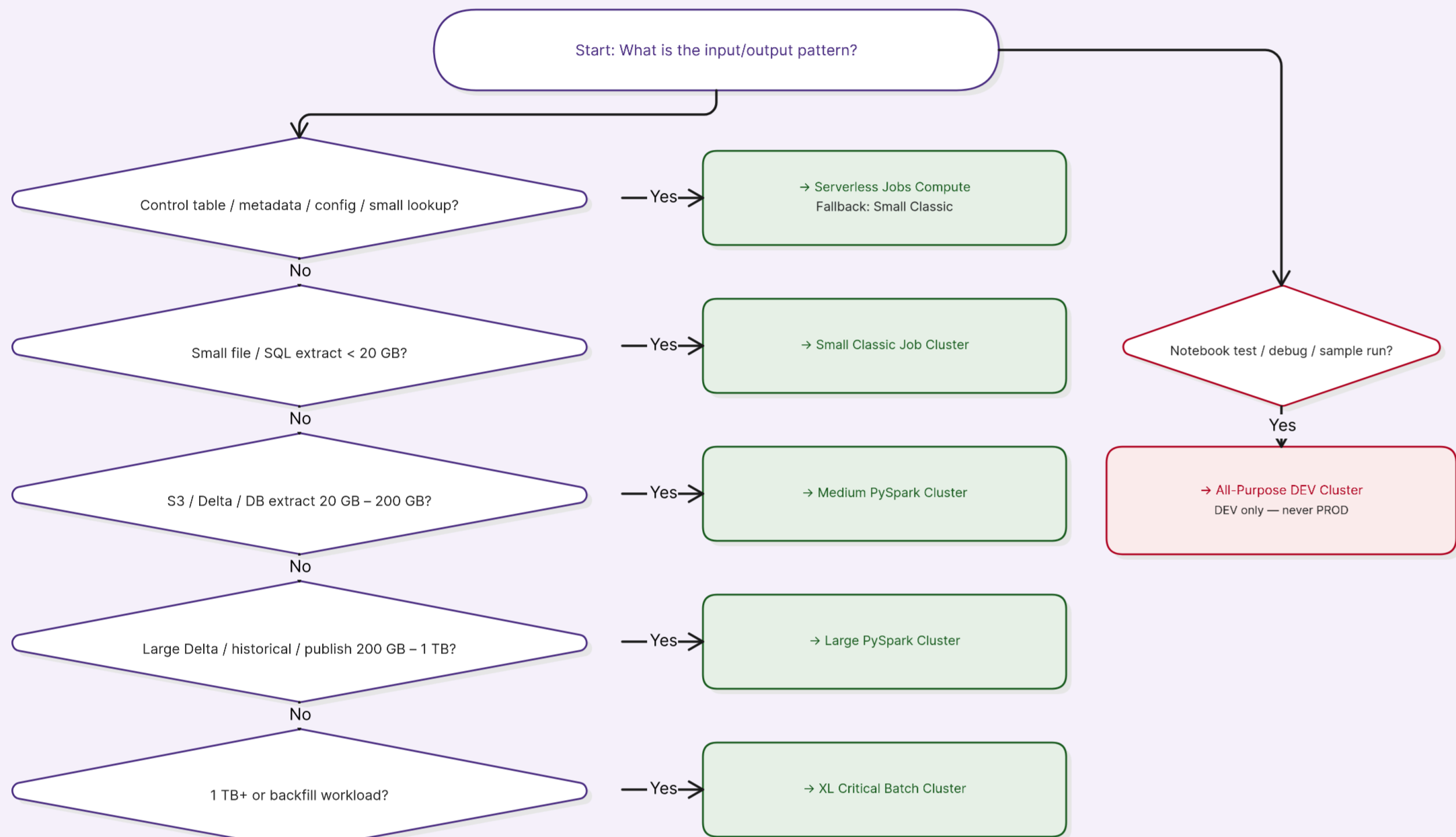
1. Medium PySpark Job Cluster
Use For:
Standard transforms, joins, aggregations, landing/pre-land
Sizing:
Driver: 8-16 vCPU, 32-64 GB
Worker: 8-16 vCPU, 32-64 GB
Autoscale: min 2 / max 6
Auto-Termination: 10-15 min

1. Large PySpark Job Cluster
Use For:
Delta merge, large joins, publish, high shuffle workloads
Sizing:
Driver: 16-32 vCPU, 64-128 GB
Worker: 16-32 vCPU, 64-128 GB
Autoscale: min 4 / max 16
Auto-Termination: 10-15 min

1. XL Critical Batch Cluster
Use For:
Historical backfill, 1 TB+, critical merge/update batch
Sizing:
Driver: 32 vCPU, 128 GB+
Worker: 32 vCPU, 128 GB+
Autoscale: min 8 / max 32
Use only after perf validation

1. All-Purpose DEV Cluster
Use For:
Dev, notebooks, debug, sample data, unit tests
Sizing:
Driver: 4-8 vCPU, 16-32 GB
Worker: 4-8 vCPU, 16-32 GB
Autoscale: min 1 / max 2-4
Auto-Termination: 30-60 min
▲ DEV only — No PROD batch

1. Cluster Selection Decision Flow (Input/Output Pattern)



1. Optimized Cluster Sizing Mat

Workload	Data	Compute	Driver Mem	Worker Mem	Workers	Autoscale	Notes
Lightweight Python Control	< 1 GB	Serverless Managed / Managed / Managed					Notes: audit, metadata, status update jobs
Python Reconciliation	1-20 GB	Small Classic	Driver 16-32 GB	Worker 16-32 GB	Workers 1-4	Autoscale Optional	Notes: custom libs, DB drivers, controlled runtime
Medium PySpark	20-200 GB	Medium PySpark Job Cluster	Driver 32-64 GB	Worker 32-64 GB	min 2 / max 6	Autoscale Enabled	Notes: standard joins, filters, aggregations
Heavy PySpark / Delta Merge	200 GB-1 TB	Large PySpark	Driver 64-128 GB	Worker 64-128 GB	min 4 / max 16	Autoscale Enabled	Notes: high shuffle, merge and publish workloads
Backfill / Critical Batch	1 TB+	XL Batch Cluster	Driver 128 GB+	Worker 128 GB+	min 8 / max 32	Autoscale Enabled	Notes: critical workloads after performance validation only
DEV / Debug / Small-Medium		All-Purpose DEV Cluster	Driver 16-32 GB	Worker 16-32 GB	min 1 / max 2-4	Autoscale Optional	Notes: DEV only, not PROD

1. Memory & Performance Planning Rules

Driver Memory
• Increase on driver OOM
• Increase for large execution plans
• Increase for heavy metadata collection

Worker Memory
• Increase for joins, aggregations
• Increase for Delta merge, shuffle-heavy
• Increase on executor OOM / disk spill

Worker Count
• Increase when CPU high, memory stable
• Increase when parallelism is low
• Increase when SLA missed by processing time

Spark Optimization
• Avoid collect() on large datasets
• Tune shuffle partitions, partition pruning
• Broadcast join for small lookups only
• Use Photon where compatible
• Compact small files; watch skew & spill

I/O Optimization
• Tune JDBC read partitioning
• Tune DB write batch size
• Track S3 read/write time
• Track input/output data size per job

1. Cluster Policies & Governance

serverless-python-policy
• Validated lightweight Python
• Approved libraries only
• Required tags
• Logs enabled
• No hardcoded secrets

small-python-policy
• Max 4 workers
• Small worker types only
• Auto-termination required
• Approved lib/drivers only
• Required tags

medium-pyspark-policy
• Autoscale min 2 / max 6
• Standard runtime
• Logs enabled
• Required tags
• Medium transformation jobs

large-pyspark-policy
• Autoscale min 4 / max 16
• Memory-optimized workers
• Restricted users / SPNs
• Required production tags
• Merge / publish / heavy jobs

dev-debug-policy
• All-purpose in DEV only
• Max 4 workers
• Auto-termination required
• No PROD batch execution

Required Tags

Mandatory Cluster Tags
project | workflow_name | job_name
environment | owner | cost_center
workload_type | criticality
Used for chargeback, cost tracking, ownership and PROD enforcement across all clusters.

1. Runtime, Libraries & Driver Management

Runtime & Library Standards
• Use Databricks Runtime LTS for production
• Keep DEV / SIT / UAT / PROD runtime versions aligned
• Use same runtime family across testing and production
• Install libraries via job config or cluster policy
• Validate MSSQL / PostgreSQL JDBC / ODBC drivers
• Avoid manual library installation on production clusters
• Use init scripts only when required
• Store cluster logs centrally
• Maintain separate cluster configs per environment
• No hardcoded secrets in notebooks or jobs

1. Cost Control & Right-Sizing

Cost Optimization Practices
• Use job clusters for production instead of all-purpose
• Enable auto-termination on all classic clusters
• Enable autoscaling for PySpark workloads
• Use fixed 1 worker for very small Python jobs
• Do not over-size clusters before SIT/UAT testing
• Use required tags for chargeback and cost tracking
• Review cluster utilization weekly
• Downsize clusters with low CPU/memory usage
• Increase size only when justified by SLA / spill / OOM
• Use instance pools only if startup time is a major bottleneck

Feedback Loop → Re-size

1. Monitoring-Based Cluster Optimization (Feedback Loop)

Metrics Captured per Job Run
• Job duration • Cluster startup time
• CPU utilization • Driver memory usage
• Executor memory usage • Shuffle read/write
• Spill to disk • Failed tasks • Skewed tasks
• Input records / output records
• Input data size / output data size
• DB read/write time • S3 read/write time
• Cost per job run

Feedback Decisions → Re-Size Clusters
• High CPU, low memory issue → increase workers
• Memory spill or executor OOM → increase worker memory
• Driver OOM → increase driver size
• Low CPU/memory utilization → downsize cluster
• Long startup time → evaluate serverless or instance pool
• Slow DB writes → tune batch size & connection parallelism
Note: Final sizing must be confirmed via SIT/UAT runtime metrics — not assumptions.

1. Final Databricks Cluster Recommendation

Adopt a workload-based compute model:
• Serverless → validated lightweight Python / control jobs • Small Classic → Python jobs needing custom libs, drivers, controlled runtime
• Medium PySpark → standard transformations • Large PySpark → Delta merge, large landing & publish workloads
• XL Batch → backfill or 1 TB+ critical workloads only • All-Purpose → DEV / debugging only (never PROD)
• Enforce cluster policies, auto-termination, autoscaling, runtime standardization, required tags & monitoring
• Right-size clusters using actual input size, output size, shuffle, memory, runtime and cost metrics